
Guía paso a paso — Calculadora Estándar

Asignatura: Laboratorio Web 1 — Tecnicatura Universitaria en Desarrollo Web.

Profesor: Jorge Ceferino Valdez.

Público objetivo: Alumnos con introducción previa a HTML5, CSS3 vanilla y JavaScript vanilla que desean consolidar sus conocimientos mediante un proyecto práctico y progresivo.

Tecnologías: HTML5, CSS3 vanilla, JavaScript vanilla (sin frameworks ni librerías).

Enfoque pedagógico: Cada tarea se apoya en la anterior, permitiendo ver la evolución de una “página estática” hasta una “aplicación web funcional”. No se introducen patrones de diseño formales; sí se muestran patrones de maquetación, manejo de estado, y una refactorización guiada (extraer funciones y centralizar el dibujado) razonada paso a paso.

Tabla de contenidos

1. Visión general del proyecto
 2. Cómo leer esta guía
 3. Tarea 1 — Página HTML estructural
 4. Tarea 2 — Estilos CSS: maquetación con Flexbox y Grid
 5. Tarea 3 — JavaScript básico: capturar números
 6. Tarea 4 — Operaciones aritméticas y estado
 7. Tarea 5 — Historial, errores y feedback visual
 8. Tarea 6 — Persistencia local con localStorage
 9. Tarea 7 — Soporte de teclado y accesibilidad
 10. Arquitectura final: decisiones clave
 11. Checklist de aprendizaje
-

1. Visión general del proyecto

Este ejercicio construye una calculadora estándar funcional en el navegador. Al finalizar las 7 tareas, la aplicación permite:

- Realizar operaciones básicas: suma, resta, multiplicación, división.
- Borrar el último dígito (DEL) o reiniciar por completo (AC).
- Cambiar signo ($\pm$) y calcular porcentaje (%).
- Manejar números decimales.
- Mostrar un historial de operación en curso (arriba del resultado).
- Persistir el historial de operaciones completadas usando `localStorage`.
- Operar desde el teclado físico en lugar de solo con el mouse.

El proyecto se compone de tres archivos, uno por cada **capa** de una página web:

Archivo	Capa	Responsabilidad
index.html	Estructura	Qué elementos existen y su semántica
estilo.css	Presentación	Cómo se ven esos elementos y su disposición espacial
script.js	Conducta y datos	Qué hace la página, cómo gestiona la información y la persistencia

La idea central de la cursada es que cada tarea **agrega una capacidad** sin romper lo anterior. La calculadora es usable desde la Tarea 1 (visualmente); lo que cambia es cuánto puede hacer.

2. Cómo leer esta guía

Cada tarea sigue siempre el mismo esquema:

1. **Objetivo** — qué problema concreto vamos a resolver.
2. **Pasos** — el código, presentado en el orden en que conviene escribirlo.
3. **Decisiones de implementación** — el porqué de **cada línea**. Esta es la parte más importante: copiar código es fácil; entender por qué está escrito así es lo que se evalúa.

Una recomendación de método: **escribí el código vos mismo**, no lo copies y pegues. Al teclear cada línea te vas a hacer preguntas (“¿por qué const y no let?”, “¿por qué usamos dataset y no id?”) y esta guía las responde justo al lado.

A partir de la Tarea 4 vas a ver una técnica nueva: **centralizar el dibujado**. Primero escribiremos el código “suelto”, luego detectaremos la repetición y finalmente lo agruparemos en funciones. Esa secuencia “primero el problema, después la solución” es deliberada.

3. Tarea 1 — Página HTML estructural

Objetivo

Crear la estructura semántica del documento, organizar los botones de la calculadora y aplicar estilos básico para que la página se vea ordenada y centrada.

Paso 1.1: Estructura HTML semántica

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Calculadora Estándar</title>
  <link rel="stylesheet" href="estilo.css">
</head>
<body>
  <main class="calculadora">
    <header>
      <h1>Calculadora</h1>
    </header>

    <section class="pantalla" aria-live="polite" aria-atomic="true">
      <div id="historial" class="historial" aria-label="Historial de operaciones"></div>
      <div id="display" class="display" aria-label="Resultado actual">0</div>
    </section>

    <section class="teclado">
      <button type="button" data-action="clear" class="btn util ac" aria-label="Borrar todo">AC</button>
      <button type="button" data-action="delete" class="btn util" aria-label="Borrar último dígito">DEL</button>
```

```

<button type="button" data-action="percent" class="btn util" aria-label="Porcentaje">%</button>
<button type="button" data-action="operator" data-value="/" class="btn operador" aria-label="Dividir">÷</button>

<button type="button" data-action="number" data-value="7" class="btn" aria-label="Siete">7</button>
<button type="button" data-action="number" data-value="8" class="btn" aria-label="Ocho">8</button>
<button type="button" data-action="number" data-value="9" class="btn" aria-label="Nueve">9</button>
<button type="button" data-action="operator" data-value="*" class="btn operador" aria-label="Multiplicar">×</button>

<button type="button" data-action="number" data-value="4" class="btn" aria-label="Cuatro">4</button>
<button type="button" data-action="number" data-value="5" class="btn" aria-label="Cinco">5</button>
<button type="button" data-action="number" data-value="6" class="btn" aria-label="Seis">6</button>
<button type="button" data-action="operator" data-value="-" class="btn operador" aria-label="Restar">-</button>

<button type="button" data-action="number" data-value="1" class="btn" aria-label="Uno">1</button>
<button type="button" data-action="number" data-value="2" class="btn" aria-label="Dos">2</button>
<button type="button" data-action="number" data-value="3" class="btn" aria-label="Tres">3</button>
<button type="button" data-action="operator" data-value="+" class="btn operador" aria-label="Sumar">+</button>

<button type="button" data-action="toggle-sign" class="btn util" aria-label="Cambiar signo">±</button>
<button type="button" data-action="number" data-value="0" class="btn" aria-label="Cero">0</button>
<button type="button" data-action="decimal" class="btn" aria-label="Punto decimal">.</button>
<button type="button" data-action="calculate" class="btn igual" aria-label="Calcular resultado">=</button>
</section>
</main>

<script src="script.js"></script>
</body>
</html>

```

Decisiones de implementación, línea por línea:

- **<!DOCTYPE html>**: Le indica al navegador que use el modo estándar de HTML5. Sin esta línea, algunos navegadores entran en “quirks mode” (modo de compatibilidad antiguo) y el CSS se interpreta de forma inconsistente.
- **<html lang="es">**: El atributo lang declara el idioma del contenido. Mejora la accesibilidad (los lectores de pantalla eligen la voz correcta) y el comportamiento del navegador (corrector ortográfico, traducción automática).
- **<meta charset="UTF-8">**: Define la codificación de caracteres. Garantiza que tildes, eñes y símbolos matemáticos ($\div$, $\times$, $\pm$) se rendericen correctamente. Va lo más arriba posible dentro del <head> para que el navegador sepa cómo decodificar el resto del archivo.
- **<meta name="viewport" ...>**: Metaetiqueta esencial para diseño responsive. Le dice al navegador que el ancho de la ventana de visualización debe coincidir con el ancho del dispositivo, a escala 1.0. Sin esto, los móviles encogen la página para simular una pantalla de escritorio.
- **<title>**: Texto que aparece en la pestaña del navegador. No es decorativo: es lo que se ve en el historial y en los marcadores.
- **<link rel="stylesheet" href="estilo.css">**: Vincula la hoja de estilos **externa**. Mantener el CSS en su propio archivo separa la presentación de la estructura: podemos cambiar todos los colores sin tocar el HTML.
- **<main class="calculadora">**: La etiqueta <main> es semántica: indica que su contenido es el principal del documento. Es útil para accesibilidad y SEO. La clase .calculadora nos servirá de “hook” para los estilos.
- **<header> + <h1>**: Encabezado de nivel 1, único en la página. Describe el contenido principal (la calculadora).
- **<section class="pantalla">**: Sección que agrupa el display y el historial. aria-live="polite" y aria-atomic="true" son atributos de accesibilidad: cuando el contenido cambia, los lectores de pantalla lo anuncian.
- **id="historial" y id="display"**: Identificadores únicos. JavaScript usará document.getElementById para localizar y actualizar estos dos elementos.
- **aria-label="..."**: Etiquetas accesibles que describen la función de cada contenedor para lectores de pantalla.
- **<section class="teclado">**: Contenedor de los 20 botones. Es una sección, no un , porque estos botones no son una lista semántica: son controles de una interfaz de cálculo.

Uso de data-action y data-value

Cada botón lleva atributos personalizados:

- **data-action="number" + data-value="7"**: Identifica “este botón ingresa el número 7”. No usamos el texto interno (textContent)

porque los símbolos matemáticos ($\div$, $\times$, $-$) no son los mismos que los operadores de JavaScript (`/`, `*`, `-`). Mapear separadamente es más robusto.

- **type="button"**: Es clave. Por defecto, un `<button>` dentro de un `<form>` actuaría como `type="submit"`. Sin formulario no hay riesgo de recarga, pero ser explícito con `type="button"` previene comportamientos inesperados si algún día movemos los controles.
- **¿Por qué no usamos `<input type="button">`?** Los botones `<button>` son más flexibles: admiten contenido HTML (por ejemplo, íconos) y son más amigables para accesibilidad.
- **aria-label="Borrar último dígito"**: Cada botón tiene un texto alternativo para lectores de pantalla. Algunos símbolos ($\div$, $\times$, $\pm$) no se pronuncian bien por sí solos; el `aria-label` corrige eso.

Paso 1.2: CSS básico centrado

```
body {
  background: linear-gradient(135deg, #1a1a2e 0%, #16213e 100%);
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  min-height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 20px;
}

.calculadora {
  background: #0f3460;
  border-radius: 16px;
  box-shadow: 0 12px 40px rgba(0, 0, 0, 0.4);
  width: 100%;
  max-width: 360px;
  padding: 20px;
  display: flex;
  flex-direction: column;
  gap: 16px;
}
```

Decisiones de implementación, línea por línea:

- **background: linear-gradient(...)**: Gradiente sutil de grises azulados. Da profundidad al fondo sin recursos externos.
- **font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif**: Lista de respaldo. Si el sistema tiene Segoe UI (Windows), la usa; si no, cae en Tahoma, luego Geneva, luego cualquier sans-serif.
- **min-height: 100vh**: La altura mínima del `<body>` es el 100 % de la altura de la ventana. Así el gradiente cubre toda la pantalla aunque el contenido sea pequeño.
- **display: flex; justify-content: center; align-items: center;**: Patrón clásico de Flexbox para centrar **perfectamente** un elemento tanto horizontal como verticalmente en su contenedor.
- **padding: 20px**: Evita que la calculadora toque los bordes de la pantalla en dispositivos pequeños.
- **.calculadora**: La “caja” de la calculadora.
 - **border-radius: 16px**: Esquinas redondeadas para un aspecto moderno.
 - **box-shadow: 0 12px 40px ...**: sombra sutil que “levanta” la caja sobre el fondo.
 - **max-width: 360px**: Limitamos el ancho para que en pantallas grandes la calculadora no se desproporcione.
 - **display: flex; flex-direction: column;**: Apila los hijos (header, pantalla, teclado) verticalmente.
 - **gap: 16px**: Espacio entre cada sección, sin necesidad de márgenes individuales.

Nota: Aún no diseñamos los botones individuales. Eso viene en la Tarea 2, donde usamos **CSS Grid** para organizar el teclado.

4. Tarea 2 — Estilos CSS: maquetación con Flexbox y Grid

Objetivo

Diseñar la pantalla de la calculadora y el teclado de 4×5 botones, aplicando colores, sombras y estados de interacción.

Paso 2.1: Reset básico y consistencia

```
*, ::before, ::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

Decisiones de implementación:

- **box-sizing: border-box:** Hace que width y height incluyan el borde y el relleno. Sin esto, si agregamos padding o border a un elemento, estos “suman” tamaño y rompen la maquetación. border-box hace que el tamaño sea predecible: el borde y el padding “comen” por dentro, no por fuera.
- **margin: 0; padding: 0;** Los navegadores aplican márgenes y paddings por defecto a muchos elementos. El reset elimina esas inconsistencias y nos da un lienzo limpio.

Paso 2.2: Pantalla (display e historial)

```
.pantalla {
  background: #1a1a2e;
  border-radius: 12px;
  padding: 12px 16px;
  text-align: right;
  display: flex;
  flex-direction: column;
  gap: 4px;
  min-height: 80px;
  justify-content: center;
}

.historial {
  color: #a0a0a0;
  font-size: 0.85rem;
  min-height: 18px;
  word-break: break-all;
}

.display {
  color: #fff;
  font-size: 2.4rem;
  font-weight: 300;
  min-height: 40px;
  word-break: break-all;
}
```

Decisiones de implementación, línea por línea:

- **.pantalla:** Contenedor oscuro que simula la “pantalla física” de una calculadora.
 - **text-align: right:** El resultado siempre se alinea a la derecha, como en calculadoras reales.
 - **flex-direction: column; gap: 4px;** Apila historial y resultado, separados por un pequeño espacio.
- **.historial:** Texto gris claro, más pequeño. Es la operación en curso (ej. 12 +).
 - **min-height: 18px:** Asegura que, aunque esté vacío, el espacio físico esté reservado. Si no, el display “saltaría” when something appears.
 - **word-break: break-all:** Si la operación es muy larga, rompe la línea en cualquier carácter para que no desborde la caja.
- **.display:** Número principal, grande y blanco.
 - **font-weight: 300:** Tipografía fina, estética moderna.
 - **word-break: break-all:** Permite que un número muy largo se parta en varias líneas sin salirse del contenedor.

Paso 2.3: Teclado — Grid de 4 columnas

```
.teclado {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  gap: 10px;  
}
```

Decisiones de implementación:

- **display: grid:** CSS Grid es la herramienta ideal para maquetar una cuadrícula bidimensional (filas y columnas). En este caso, 4 columnas de igual ancho.
- **grid-template-columns: repeat(4, 1fr):** Crea 4 columnas, cada una ocupando 1fr (una fracción del espacio disponible). Todas miden lo mismo: 25 % cada una.
- **gap: 10px:** Espacio entre botones, sin afectar los márgenes externos del teclado.

¿Por qué Grid y no Flexbox? Flexbox es ideal para distribuir elementos en **una dimensión** (una fila o una columna). Para una tabla de 4×5, Grid es más sencillo y declarativo. Usamos Flexbox para centrar el <body> (una fila única) y para apilar verticalmente la .pantalla; Grid para la matriz de botones.

Paso 2.4: Botones base y variantes

```
.btn {  
  background: #16213e;  
  color: #fff;  
  border: none;  
  border-radius: 10px;  
  font-size: 1.3rem;  
  padding: 16px 0;  
  cursor: pointer;  
  transition: background 0.15s ease, transform 0.05s ease;  
  font-family: inherit;  
}  
  
.btn:hover { background: #1a3a5c; }  
.btn:active { transform: scale(0.96); background: #0f2b4a; }  
  
.btn.util { background: #533483; }  
.btn.util:hover { background: #6b4aa3; }  
  
.btn.operador { background: #e94560; font-size: 1.5rem; }  
.btn.operador:hover { background: #ff6b81; }  
  
.btn.igual { background: #00b894; font-size: 1.6rem; }  
.btn.igual:hover { background: #00d9a5; }  
  
.btn.ac { background: #c0392b; }  
.btn.ac:hover { background: #e74c3c; }
```

Decisiones de implementación, línea por línea:

- **.btn:** Selector base que aplica a **todos** los botones.
 - **border: none; border-radius: 10px;** Elimina el borde nativo y redondea las esquinas.
 - **cursor: pointer;** Cambia el cursor a una mano, comunicando que el elemento es interactivo.
 - **transition: background 0.15s ease, transform 0.05s ease;** Animación suave de color y escala. 0.15s para el fondo es perceptible pero ágil; 0.05s para el “click” es casi instantáneo.
 - **font-family: inherit;** Fuerza al botón a usar la misma fuente que el resto del documento, en lugar de la fuente por defecto del sistema para botones.
- **:hover y :active:**
 - **:hover** responde cuando el mouse está sobre el botón. **:active** responde en el momento exacto del clic.
 - **transform: scale(0.96)** reduce ligeramente el botón al clickear, dando un feedback físico de “presión”.

■ Variantes por clase:

- `.util` (AC, DEL, %, ±): color violeta. Distingue las funciones de “utilidad” de los números.
- `.operador` (+, −, ×, ÷): color rojo-rosa. Es la convención visual de calculadoras: los operadores destacan.
- `.igual` (=): color verde. Indica “acción final” o “resultado”.
- `.ac` (borrar todo): rojo puro. Señala “peligro” o “destructivo”.

Paso 2.5: Responsive para móviles

```
@media (max-width: 400px) {  
  .calculadora { padding: 14px; border-radius: 12px; }  
  .btn { font-size: 1.1rem; padding: 14px 0; }  
  .display { font-size: 2rem; }  
}
```

Decisiones de implementación:

- `@media (max-width: 400px)`: Bloque de estilos que solo se aplican si la pantalla mide 400px o menos de ancho.
- Reducimos padding y font-size para que los botones sigan siendo usables en pantallas pequeñas.

5. Tarea 3 — JavaScript básico: capturar números

Objetivo

Conectar los botones numéricos con el display, permitiendo que el usuario ingrese dígitos y vea el resultado en pantalla.

Paso 3.1: Capturar referencias y evento de delegación

```
const display = document.getElementById('display');  
const teclado = document.querySelector('.teclado');  
  
teclado.addEventListener('click', (e) => {  
  const btn = e.target.closest('button');  
  if (!btn) return;  
  
  const action = btn.dataset.action;  
  
  if (action === 'number') {  
    const num = btn.dataset.value;  
    // ... lógica de ingreso de número ...  
  }  
});
```

Decisiones de implementación, línea por línea:

- `const display = document.getElementById('display')`: Tomamos la referencia al display una sola vez. Buscamos por id porque es la forma más rápida de localizar un único elemento (el id debe ser único en el documento).
- `const teclado = document.querySelector('.teclado')`: Usamos `querySelector` para capturar el contenedor del teclado por su clase. Es el primer (y único) elemento con `.teclado`.
- `teclado.addEventListener('click', (e) => { ... })`: Delegación de eventos. En lugar de asignar 20 listeners (uno por botón), asignamos **uno solo** al contenedor padre. Cualquier clic que “burbujea” hacia arriba desde un botón será capturado aquí.
 - **Ventaja de delegación**: menos memoria, código más corto, y funciona automáticamente si agregamos/quitamos botones dinámicamente.
- `e.target.closest('button')`: `e.target` puede ser el texto interno del botón. `closest('button')` sube por el DOM hasta encontrar el `<button>` más cercano. Si el clic fue fuera de un botón, devuelve `null`.
- `if (!btn) return;`: Guarda defensiva. Si no se encontró un botón, no hacemos nada.

- `const action = btn.dataset.action;` Los atributos `data-*` del HTML son accesibles vía `dataset`. `data-action="number"` se convierte en `btn.dataset.action` con valor `"number"`. Es una forma limpia de guardar “metadatos” en un elemento sin usar `id`, clases o atributos no estándar.

Paso 3.2: Ingresar números con una variable de estado

```
let current = '0';

function inputNumber(num) {
  if (current === '0') {
    current = num;
  } else {
    current += num;
  }
  render();
}

function render() {
  display.textContent = current;
}
```

Decisiones de implementación, línea por línea:

- `let current = '0'`: Guardamos el valor actual como **string**, no como número. ¿Por qué string?
 1. Permitimos concatenación directa (`"12" + "3" = "123"`).
 2. Facilita el manejo del punto decimal (`"5."` es válido como string; como número sería 5, perdiendo la intención del usuario).
 3. Permite borrar dígito por dígito con `slice(0, -1)`.
- `if (current === '0') { current = num; }`: Si el display muestra 0, el primer dígito reemplaza el cero. Si no, concatena.
- `render()`: Función centralizada. En vez de tocar `display.textContent` en cada acción, tenemos **una sola función** que dibuja el estado. Si mañana cambiamos cómo se muestra (por ejemplo, agregamos separadores de miles), se modifica solo `render()`.

Patrón que arrastramos: `render()` será la “puerta única” hacia el display. Cada función que modifica el estado llama a `render()` al final. Es equivalente a la función `renderTasks()` del proyecto anterior.

6. Tarea 4 — Operaciones aritméticas y estado

Objetivo

Implementar las cuatro operaciones básicas, guardando el primer operando y el operador hasta que el usuario presione “=”.

Paso 4.1: Variables de estado ampliadas

```
let current = '0';
let previous = null;
let operator = null;
let shouldReset = false;
```

Decisiones de implementación:

- **previous**: Guarda el primer número de la operación. Ejemplo: si el usuario escribe `12 + 3`, `previous` guarda 12.
- **operator**: El símbolo del operador pendiente (`+`, `-`, `*`, `/`). Si es `null`, no hay operación pendiente.
- **shouldReset**: Bandera booleana. Se activa cuando el usuario acaba de presionar un operador o “=”.

Ejemplo de flujo: Usuario teclea 5, +, 3, =.

- 5 → `current = "5"`

- + → `previous = 5, operator = "+", shouldReset = true`

- 3 → como shouldReset es true, current se reinicia: "3"
- = → calcula previous + current → 8, y muestra el resultado.

Paso 4.2: Ingresar un operador

```
function inputOperator(op) {
  // Si hay operación pendiente, primero resolvemos
  if (operator !== null && !shouldReset) {
    ejecutarCalculo();
  }

  previous = parseFloat(current);
  operator = op;
  shouldReset = true;
}
```

Decisiones de implementación, línea por línea:

- **if (operator !== null && !shouldReset):** Si el usuario presiona un operador inmediatamente después de otro (ej. 5 + * 3), primero calculamos el resultado parcial (5 +) antes de guardar el nuevo operador.
- **parseFloat(current):** Convertimos la cadena del display a número real. parseFloat acepta decimales (a diferencia de parseInt).
- **shouldReset = true:** Prepara la bandera para que el próximo dígito reemplace el display.

Paso 4.3: Calcular el resultado

```
function ejecutarCalculo() {
  if (previous === null || operator === null) return;

  const b = parseFloat(current);
  let resultado = calcular(previous, b, operator);

  if (!isFinite(resultado)) {
    current = 'Error';
  } else {
    resultado = parseFloat(resultado.toFixed(10));
    current = String(resultado);
  }

  operator = null;
  previous = null;
  shouldReset = true;
}

function calcular(a, b, op) {
  switch (op) {
    case '+': return a + b;
    case '-': return a - b;
    case '*': return a * b;
    case '/': return (b === 0) ? Infinity : a / b;
    default: return b;
  }
}
```

Decisiones de implementación, línea por línea:

- **if (previous === null || operator === null) return;** Guarda defensiva. No podemos calcular si falta un operando u operador.
- **parseFloat(resultado.toFixed(10)):** JavaScript tiene errores de precisión con decimales (ej. 0.1 + 0.2 = 0.30000000000000004). toFixed(10) fija 10 decimales; parseFloat elimina ceros finales innecesarios.
- **!isFinite(resultado):** Detecta Infinity o NaN. Si ocurre (por ejemplo división por cero), mostramos 'Error'.
- **String(resultado):** El resultado se guarda como cadena, manteniendo consistencia con current.

- `operator = null; previous = null;`: Limpia el estado de operación pendiente. Ahora la calculadora está lista para una operación nueva.

7. Tarea 5 — Historial, errores y feedback visual

Objetivo

Mostrar la operación en curso en un historial superior, manejar la entrada de decimales, borrar dígitos y cambiar signo.

Paso 5.1: Mostrar operación en curso

```
const historialEl = document.getElementById('historial');

function render() {
  display.textContent = current;

  if (previous !== null && operator !== null) {
    historialEl.textContent = `${previous} ${operator}`;
  } else {
    historialEl.textContent = '';
  }
}
```

Decisiones de implementación:

- `**historialEl.textContent = `${previous} ${operator}``: Muestra algo como `12 +` arriba del resultado actual. Es el mismo patrón de calculadoras físicas.
- El historial se limpia automáticamente cuando no hay operación pendiente.

Paso 5.2: Punto decimal

```
function inputDecimal() {
  if (shouldReset) {
    current = '0.';
    shouldReset = false;
    return;
  }
  if (!current.includes('.')) {
    current += '.';
  }
}
```

Decisiones de implementación:

- `if (current.includes('.'))`: Evita que el usuario ponga múltiples puntos decimales (ej. `"5..3"`).
- `if (shouldReset)`: Si acabamos de presionar `"="` u otro operador, el siguiente punto decimal reemplaza el display por `"0."`.

Paso 5.3: DEL y ±

```
function deleteLast() {
  if (current === 'Error' || shouldReset) {
    clearAll();
    return;
  }
  if (current.length === 1 || (current.length === 2 && current.startsWith('-'))) {
    current = '0';
  } else {
    current = current.slice(0, -1);
  }
}
```

```

}

function toggleSign() {
  if (current === '0') return;
  current = current.startsWith('-') ? current.slice(1) : '-' + current;
}

```

Decisiones de implementación:

- **deleteLast**: Si el display está en “Error”, DEL funciona como AC.
- **current.slice(0, -1)**: Corta la última letra del string. Es una ventaja de manejar current como cadena.
- **toggleSign**: Simplemente agrega o quita el signo - al principio de la cadena.

Paso 5.4: Borrar todo (AC)

```

function clearAll() {
  current = '0';
  previous = null;
  operator = null;
  shouldReset = false;
}

```

Decisiones de implementación:

- Vuelve **todas** las variables de estado a sus valores iniciales. Es una función “reset completo”.

8. Tarea 6 — Persistencia local con localStorage

Objetivo

Guardar el historial de operaciones completadas en localStorage, para que persistan entre recargas de página.

Paso 6.1: Guardar operaciones

```

const STORAGE_KEY = 'calculadora_historial';

function guardarEnHistorial(entry) {
  const historial = cargarHistorial();
  historial.unshift(entry); // más reciente primero
  if (historial.length > 10) historial.pop(); // máximo 10 entradas
  localStorage.setItem(STORAGE_KEY, JSON.stringify(historial));
}

function cargarHistorial() {
  const data = localStorage.getItem(STORAGE_KEY);
  return data ? JSON.parse(data) : [];
}

```

Decisiones de implementación, línea por línea:

- **const STORAGE_KEY = 'calculadora_historial'**: Una constante con el nombre de la clave. Si algún cambia, se toca en un solo lugar.
- **localStorage.getItem(...)** / **localStorage.setItem(...)**: API síncrona de almacenamiento persistente en el navegador. Los datos sobreviven cierres de pestaña y reinicios del navegador.
- **JSON.stringify** / **JSON.parse**: localStorage solo almacena strings. Convertimos el arreglo a texto (stringify) al guardar, y de texto a arreglo (parse) al leer.
- **historial.unshift(entry)**: Agrega la nueva operación al **inicio** del arreglo (más reciente primero).
- **if (historial.length > 10) historial.pop()**: Mantiene solo las últimas 10 operaciones. Evita que el almacenamiento crezca indefinidamente.

Paso 6.2: Integrar con ejecutarCalculo

```
function ejecutarCalculo() {  
  // ... código de cálculo ...  
  guardarEnHistorial(`${previous} ${operator} ${b} = ${current}`);  
  // ... limpiar estado ...  
}
```

Decisiones de implementación:

- Se invoca dentro de ejecutarCalculo, **después** de obtener el resultado (current) y **antes** de limpiar operator y previous. Así se guarda la operación completa.

9. Tarea 7 — Soporte de teclado y accesibilidad

Objetivo

Permitir que el usuario escriba con el teclado físico, no solo clicando botones.

Paso 7.1: Escuchar teclas del documento

```
document.addEventListener('keydown', (e) => {  
  const key = e.key;  
  
  if (/^[0-9]$/.test(key)) {  
    e.preventDefault();  
    inputNumber(key);  
    render();  
    return;  
  }  
  
  switch (key) {  
    case '.':  
    case ',':  
      e.preventDefault(); inputDecimal(); break;  
    case '+':  
      e.preventDefault(); inputOperator('+'); break;  
    case '-':  
      e.preventDefault(); inputOperator('-'); break;  
    case '*':  
    case 'x':  
    case 'X':  
      e.preventDefault(); inputOperator('*'); break;  
    case '/':  
      e.preventDefault(); inputOperator('/'); break;  
    case 'Enter':  
    case '=':  
      e.preventDefault(); ejecutarCalculo(); break;  
    case 'Escape':  
      e.preventDefault(); clearAll(); break;  
    case 'Backspace':  
      e.preventDefault(); deleteLast(); break;  
    case '%':  
      e.preventDefault(); inputPercent(); break;  
    default:  
      return;  
  }  
  
  render();  
});
```

Decisiones de implementación, línea por línea:

- `document.addEventListener('keydown', ...)`: Escucha **todas** las teclas del documento. El foco no importa: funciona aunque el usuario no haya hecho clic en ningún input.
- `e.preventDefault()`: Cancela el comportamiento por defecto de algunas teclas. Por ejemplo, Enter no debe enviar un formulario inexistente, ni / abrir el buscador de atajos del navegador.
- `/^[0-9]$/.test(key)`: Expresión regular que verifica si la tecla es un dígito del 0 al 9. Es más compacto que 10 case.
- `case 'x': case 'X':`: Aceptamos tanto * como * como X para multiplicación, ya que algunos usuarios asocian la letra X con “por”.
- `return` en default: Si la tecla no está mapeada, no llegamos al `render()` final.

10. Arquitectura final: decisiones clave

10.1 Separación de responsabilidades

Capa	Rol	Archivo
Estructura	Define qué elementos existen y su semántica	<code>index.html</code>
Presentación	Define cómo se ven y se comportan visualmente	<code>estilo.css</code>
Conducta y datos	Maneja la lógica, el estado y la persistencia	<code>script.js</code>

10.2 Modelo y Vista: un flujo en una sola dirección

En lugar de tocar el DOM “aquí y allí”, el proyecto adopta un flujo **unidireccional**:

El usuario interactúa (clic o teclado)

↓

Se modifican las variables de estado (modelo)

↓

`render()` actualiza el display e historial (vista)

↓

Al completar una operación:

`guardarEnHistorial()` escribe en `localStorage` (persistencia)

Ventajas, pedagógicas y técnicas:

- **Predecibilidad**: siempre sabés dónde está la verdad — en las variables `current`, `previous`, `operator`.
- **Depuración fácil**: un `console.log({ current, previous, operator })` en cualquier momento te muestra el estado real de la app.
- **Extensibilidad**: si mañana querés agregar “memoria” (guardar/recuperar un valor), agregás una variable `memory` y un par de funciones. Nada más.

10.3 Por qué `render()` es una sola función

Toda actualización visual pasa por `render()`. Significa que:

- Existe **una sola puerta** hacia el DOM del display y el historial. Ningún otro código toca `display.textContent` directamente.
- Las funciones (`inputNumber`, `inputOperator`, `ejecutarCalculo`, etc.) son cortas: modifican el estado y delegan el dibujado.
- El formato (separadores de miles) se aplica en un solo lugar.

10.4 Delegación de eventos sobre 20 listeners

Usamos **un solo** `addEventListener` en el contenedor `.teclado`, en lugar de 20 listeners individuales:

- Menos memoria consumida.
- El código es más corto y limpio.
- Si se agregaran botones dinámicamente, no necesitaríamos nuevos listeners.

10.5 Estado como string (current)

Aunque estamos hablando de números, decidimos manejar current como **string**:

- Facilita la concatenación ("12" + "3" = "123").
 - Permite manejar "5." sin perder el punto visual.
 - Permite borrar dígito por dígito.
 - Solo se convierte a Number en el momento del cálculo (parseFloat).
-

11. Checklist de aprendizaje

Al finalizar esta guía, el alumno debería ser capaz de explicar y reproducir:

- ☐ Estructura semántica de un documento HTML5 con lang, charset, viewport y separación de archivos.
 - ☐ Uso de data-* atributos para guardar metadatos en elementos HTML sin recurrir a id.
 - ☐ Maquetación con CSS Grid para cuadrículas bidimensionales (teclado 4×5).
 - ☐ Maquetación con CSS Flexbox para distribución en una dimensión (pantalla centrada).
 - ☐ Delegación de eventos: un solo listener en un contenedor padre.
 - ☐ Manejo de estado con variables (current, previous, operator, shouldReset) y por qué current es string.
 - ☐ Patrón “modificar estado → llamar a render()” para separar lógica de presentación.
 - ☐ Validación defensiva: no agregar tareas vacías, no dividir por cero, no múltiples puntos decimales.
 - ☐ Persistencia en el navegador con localStorage, JSON.stringify y JSON.parse.
 - ☐ Soporte de teclado físico mediante keydown y preventDefault.
 - ☐ Accesibilidad básica: aria-label, aria-live, aria-atomic.
-

Apéndice: estructura de archivos final

calculadora-estandar/

— index.html	→ Estructura y controles de la interfaz
— estilo.css	→ Reglas de presentación y estados visuales
— script.js	→ Lógica, modelo de datos, eventos y persistencia
— GUIA_CALCULADORA.md	→ Este documento

Consejo para practicar. Replicá el proyecto desde cero con esta guía cerrada, y abrila solo cuando te bloquee. La repetición activa —“hacer sin mirar”— es la que consolida el aprendizaje. Y un ejercicio extra muy recomendable: una vez que tu versión funcione, intentá explicarle en voz alta a un compañero **por qué render() es una sola función**. Si podés explicarlo, lo entendiste.